SEETRUE

SeeTrue Client API

V2.0
01.02.2023

Introduction **2**
Architecture overview **2**
Opening ports **4**
Communicating with the server **5**
The eye-tracker devices list 5
Set Eye Tracker device 6
Stop SeeTrue Server 6
Stop Picture Processing 7
Set the Width and Height of the user screen 7
Run Picture Processing 8
Start Eye-Pictures Stream 9
Stop Eye-Pictures Stream 9
Calibration 9
Recalibrate 10
Get the Calibration Point from the server 11
Add Calibration Point 11
Save Calibration Data 12
Validation 12
Receive general data from the server **12**
Receive eye data from the server 15
Receive eye-pixel frames from the server 17
Eye-event listeners **19**
The eye-server alive message 22
Add-on functionalities **23**
Marker tracking 23
Custom scene camera/no scene camera 23
Appendix 1. 24

Introduction

The SeeTrue EyeTracker system is based on the client-server architecture. The server is represented by the executable SeeTrueEyeServer.exe. We also provide two clients: SeeTruePlayer and the SeeTrueLauncher. *SeeTruePlayer* is a full application for the eye-tracking studies, data collection and visualization. *SeeTrueLauncher* is a light application that runs in the background and performs routines such as calibration and validation, and then lets third party applications simply use the eye-data stream. Typically, both server and client application(s) work on the same computer in the same Windows 10/11(x64) operation system. However, for flexibility, or to keep more computational power for user applications, you can run the server on one computer and the client on the second. In this case, you should establish a network connection between those computers (see below). For example, mobile phone client implementations can benefit from a networked server solution.

The SeeTrue EyeServer can be controlled from a user application using the SeeTrue API. For example, the SeeTrue clients use the SeeTrueAPI to communicate with the server during calibration and data acquisition. The current document describes the SeeTrueAPI in detail. It's also possible to build your own OS independent clients for the SeeTrueEye server using the SeeTrue API for controlling the eye tracking functionality.

Architecture overview

The SeeTrueEye server works as a command-line Windows x64 application. There are two ways to execute the server:

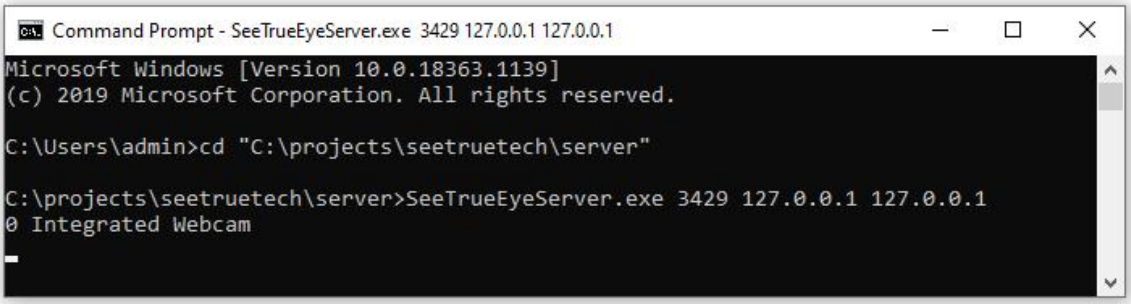
1. as a stand-alone application and connect to it remotely from a client, or
2. as a dependent application run directly from the client user interface.

To run the stand-alone application (option 1), you should use the Command Prompt (or shell terminal) and navigate to the folder with SeeTrueEyeServer.exe file and execute it with three compulsory parameters:

port number, server IP address and client IP address

For example, in case you run the server (as stand-alone) and the client on the same computer

```
C:\SeeTrueTracker> SeeTrueEyeServer.exe 3429 127.0.0.1 127.0.0.1
```



Depending on the version of the SeeTrueEye server you will see some technical information printed as the program starts. Finally, the execution should end up with a blinking cursor in the command prompt. This indicates that the server is running and waiting for a connection. On some systems, you need to allow the execution of the server application, for example due to an anti-virus application blocking the exe file from running.

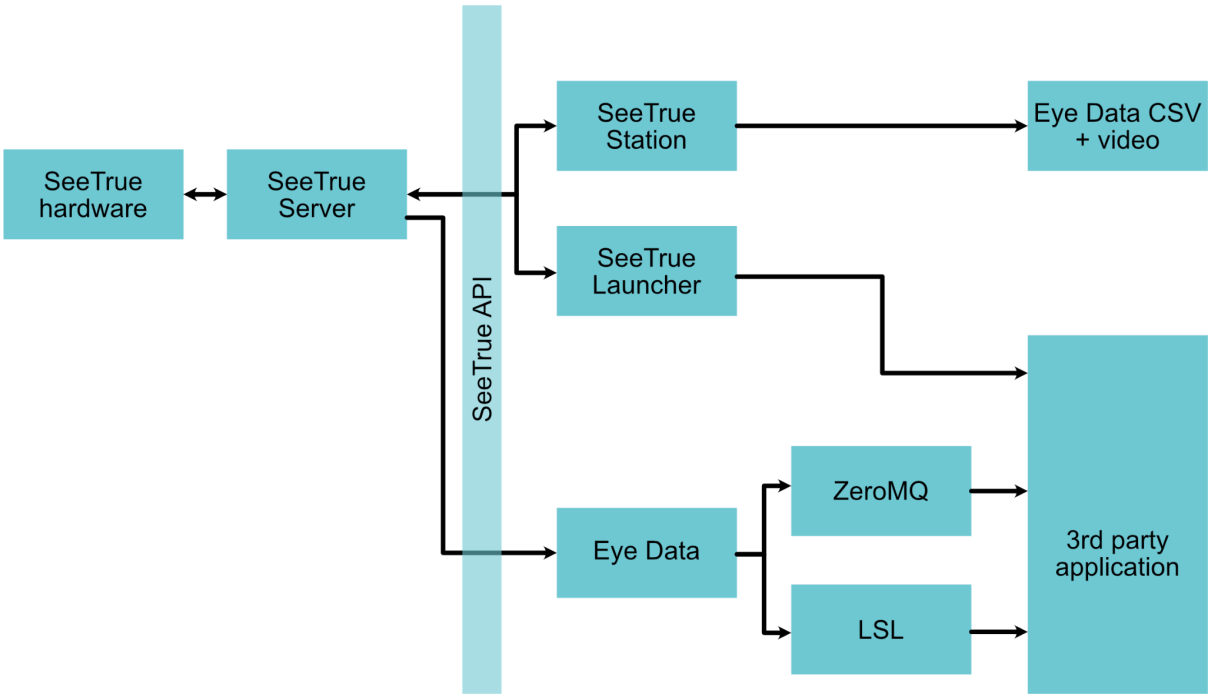


Figure 1. Schematic of the SeeTrue software structure. The server is responsible for communicating with the tracking hardware and supplying data and control through the API. SeeTrue Station is a self-contained client for controlling recordings. SeeTrue Launcher helps with setting up the connections and deals with calibration, allowing the 3rd party applications easy access to the tracking functionality. The server supplies Eye Data to the clients through a ZeroMQ interface, and through a Lab Streaming Layer (LSL) connection.

Opening ports

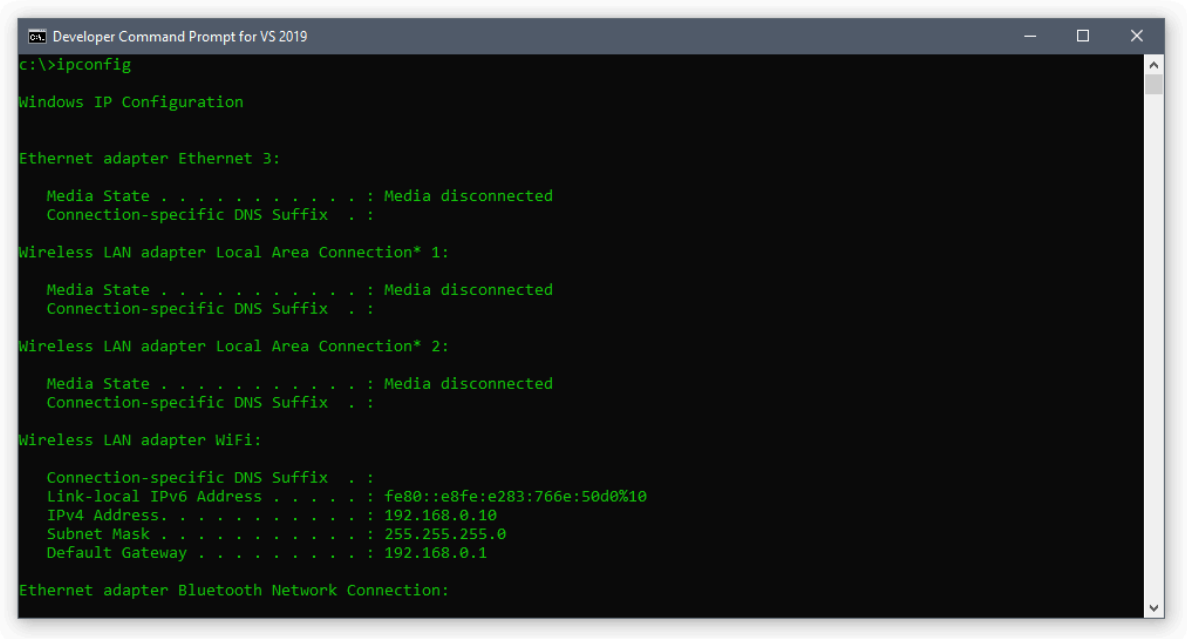
You are free to configure a different port and address from the one used in our example. The port should be free from other connections and applications. The communication with the client uses several ports for data streaming. We define the addresses of those ports with respect to the initial port number provided when running the server. The port addressing logic is shown below. Let's say that you run the server with the default port address: 3429

```
C:\Projects\SeeTrueTechnology\demo>SeeTrueEyeServer.exe 3429
127.0.0.1 127.0.0.1
```

The server will listen to the initial port 3429 for client messages. The responses will be sent back to the client on several ports according to the specification. The port for receiving general data on the client is 3430 (initial port address + 1). This port is used for general purposes, for example, to get the eye-tracker devices list, eye-tracker types, and calibration messages. The next six ports are used to receive eye-tracker data from the server:

1. Eye-Data Port: 3428 (initial port address - 1),
2. *Legacy*: *Left Eye Image Port: 3427 (initial port address - 2),*
3. *Legacy*: *Right Eye Image Port: 3426 (initial port address - 3),*
4. Scene Image Port: 3425 (initial port address - 4),
5. Calibration Picture Port: 3424 (initial port address - 5),
6. Eye-event Listeners Port: 3423 (initial port address - 6).

If you run the server with your own port number, all other ports will be addressed accordingly. All ports should be available for connection. Please check the firewall and antivirus software as they normally block all network connections by default.
If you run the server and client on the same computer, use the localhost IP address 127.0.0.1. If you run the client on a different computer, please find the IP addresses of both computers and use it to run the server. You can find out the IP address of each computer for instance through the command prompt command *ipconfig* to get the IP address of your Windows computer. Here we show an example of how to use the ipconfig command. From the output you can find the IPv4 Address: 192.168.0.10



Communicating with the server

When your server is running, you can connect to it from the client. The SeeTrue API is using several protocols for client-server communication:

1. UDP protocol to send commands from the client to the server. This messaging takes care of general control and the calibration,
2. ZeroMQ messaging for the picture stream, eye-data stream, and sending responses from the server to the client,

The API messages are formatted as JSON messages. Each client-request is a JSON string that has at least one field titled action. The minimum JSON structure is shown below:

```
{
    "action": "sendEyeTrackerTypeData"
}
```

When the SeeTrueEye server receives such a valid JSON string, it parses it and executes the action accordingly.

The eye-tracker devices list

To request the eye-tracker devices list, you should send a message with the following JSON string:

```
{
    "action": "sendEyeTrackerTypeData"
}
```

NodeJS example:

```
const dgram = require('dgram');
UDPServerControl = dgram.createSocket('udp4');
var dataToSend = {};
dataToSend.action = "sendEyeTrackerTypeData";
var message = new Buffer.from(JSON.stringify(dataToSend));
UDPServerControl.send(message,  0,  message.length,  3429,  "127.0.0.1",  function
(err){
        if (err != null) {
                console.log(err);
         }
});
```

In this example we use the 'dgram' module to send the UDP request to the server. Then we create the UDPServerControl as a udp4 socket. We use a minimum version of the message and create the JSON with only one action field. Finally, we convert the JSON string to a buffer and send it to the server. You can also add a callback function in case an error occurs when handling the API event call.

Set Eye Tracker device

After selecting the *device_type* with *sendEyeTrackerTypeData* above, one should send the command to set the eye tracking device to be used:

```
{
      "action": "setEyeTrackerDevice",
      "eyeTrackerDevice" : device_type
}
```

, where *device_type* is the string identifier for the device returned from the *sendEyeTrackerTypeData* call. This needs to be set before starting the server with *runPictureProcessing*.

Stop SeeTrue Server

To stop the eye-tracker, you should send a message with the following JSON string:

```
{
    "action": "stopServer"
}
```

NodeJS example:

```
var dataToSend = {};
dataToSend.action = "stopServer";
var message = new Buffer.from(JSON.stringify(dataToSend));
```

```
UDPServerControl.send(message, 0, message.length, 3429, “127.0.0.1”, function (err)
{
        if (err != null) {
                console.log(err);
         } else {
                UDPServerControl.close();
                 UDPServerControlReceiver.close();
                UDPServerScenePicture.close();
                UDPServerRightEyePicture.close();
                UDPServerLeftEyePicture.close();
                UDPServerEyeData.close();
        }
});
```

In this example we assume we already created UDPServerControl as udp4 socket from the 'dgram' module. We use a minimum version of the request and create the JSON with only one action field. Finally, we convert the JSON string to a buffer and send it to the server. This is also a good place to close all connections if you have opened any.

Stop Picture Processing

To stop the picture processing while keeping the eye-tracker still running, you should send a message with the following JSON string:

```
{
    "action": "stopPictureProcessing"
}
```

NodeJS example:

```
var dataToSend = {};
dataToSend.action = "stopPictureProcessing";
var message = new Buffer.from(JSON.stringify(dataToSend));
UDPServerControl.send(message, 0, message.length, 3429, “127.0.0.1”, function (err)
{
        if (err != null) {
                console.log(err);
        }
});
```

In this example we assume we already created UDPServerControl as udp4 socket from the 'dgram' module. We use a minimum version of the request and create the JSON with only one action field. Finally, we convert the JSON string to a buffer and send it to the server.

Set the Width and Height of the user screen

In case the eye-tracker is running with an external scene view like an AR device, it does not use the front (scene) camera as with the eye tracking glasses. The default values are set in the device profile but to scale the output coordinates according to the device configuration, the client may set the size of the working screen (in pixels) before running the picture processing:

```
{
      "action":"setSceneSize",
      "sceneWidth": 1920,
      "sceneHeight": 1080
}
```

You may also check if the setSceneSize action is available for the current device:

```
{
      "action":"isSceneCamAvailable"
}
```

And then wait for the Notification with the field *isValidationAvailable* which is *true* or *false*:

```
{
    "message": {
          "isValidationAvailable": true
    },
    "code": "14"
}
```

Run Picture Processing

To run the picture processing, you should send a message with the following JSON string:

```
{
      "action":"runPictureProcessing",
      "executableStreams": {
            "recEyeData": true,
             "recLeftEyeData": true,
             "recRightEyeData": true,
             "recSceneData": true
       },
       "eyeTrackerType": "device_specific_identifier",
       "calibrationData": ""
}
```

To run picture processing, you should specify several parameters:

1. *calibrationData* This is a string with the path to calibration data that was saved in a file before. You can save calibration data to a file and reuse it later.
2. *action* This is the action name that should be executed on the server.
3. *executableStreams* This is a list of streams that the client plans to receive. For example, if the client does not need the eye streams, it should be switched off (set to false). *The availability of these streams depends on the license*.

4. *eyeTrackerType* This is a type of eye-tracker to use. This should contain the name of the device supplied when you make a request for listing the available eye tracker types.

Start Eye-Pictures Stream

This functionality is subject to the license admitting eye image streams. To request the eye-picture frames during the eye-tracking, you could send a message on a fly to the server with the following JSON string:

```
{
      "action":"startEyePicturesStream"
}
```

Stop Eye-Pictures Stream

To stop the eye-picture stream, you could send a message on a fly to the server with the following JSON string:

```
{
      "action":"stopEyePicturesStream"
}
```

Calibration

People's eyes are different and the system should be calibrated accordingly. Due to SeeTrue's calibration model, the end-user can calibrate only once and then load the calibration profile for future use. We provide several options to perform the calibration:

- ***Automatic with markers.*** Here the calibration window will be presented on the client side, which means the server could be executed remotely. The coordinates of the calibration points will be sent from the server to the client. We use markers to help to locate the user's head to the best position and to automatically identify the calibration targets. The server sends the messages accordingly.
- ***Automatic without markers.*** Here the calibration window will be presented on the client side, which means the server could be executed remotely. The coordinates of the calibration points will be sent from the server to the client. There are no markers during this type of calibration. This type of calibration is best for the VR/AR applications.
- ***Manual calibration.*** Here we use the scene image as a canvas for the calibration points. The client provides the points for the server. The second person is normally involved in this calibration process who asks the end-user to look at some point in the environment and clicks on that point inside the scene image. That mouse click forms

the calibration point for the server. All rendering happens on the client side and the server could be executed remotely.

Recalibrate

This request will drop off the current calibration parameters and restart the calibration process.

To request *The Automatic with markers* (re)calibration, you should send a message with the following JSON string:

```
{
      "action":"recalibrate",
      "isAutoCalibration":true,
      "screenWidthInMM":300,
      "screenHeightInMM":240,
      "isClientMarkers":true,
}
```

The `"isAutoCalibration":true` parameter allows the server to control the calibration so the server will make the decision about the location of the next calibration point and when it should happen. The recalibration window has four graphical markers that we use to help the system to localize the calibration targets in relation to the user's head position. We provide the markers representation in the JavaScript code in the Appendix 1. We also need to know the physical size of the screen. Two parameters represent that in millimeters: `screenWidthInMM` and `screenHeightInMM`

To request *The Automatic without markers* (re)calibration, you should send a message with the following JSON:

```
{
      "action":"recalibrate",
      "isAutoCalibration":true,
      "isClientNoneMarkers":true
}
```

To request *The manual* (re)calibration, you should send a message with the following JSON:

```
{
      "action":"recalibrate",
      "isAutoCalibration":false
}
```

Get the Calibration Point from the server

During the two automatic calibration modes, the server sends the calibration points to the client via a Notification. The client should render the calibration stimuli accordingly. The structure of these messages is as follows:

```
{
    "message": {
          "x": 0.8,
          "y": 0.8
    },
    "code": "9"
}
```

For more details on *Notifications*, see section: *Receive general data from the server*.

Add Calibration Point

When calibrating manually, to add a calibration point, you should send a message with the following JSON:

```
{
     "action":"addCalibrationPoint",
     "pointX":0.5216,
     "pointY":0.7812,
     "id" : 8
}
```

Following the calibration procedure, we need to provide information about the X and Y coordinates of the calibration targets in the provided calibration image. The coordinates are relative float numbers with 0.0 - 1.0 limit. The source field is the ID for the gaze target location. We keep the calibration procedure scalable so potentially you could use different sources for calibration. Also, the client could track if the calibration point was accepted by the server. For that purpose the client generates the point ID and sends that as well. Then the server will send the calibration notification where the point's ID is used together with information about whether that point is accepted or not.

Save Calibration Data

You can request the current calibration data from the server with the following JSON message:

```
{
      "action":"saveCalibrationData"
}
```

This request will generate a calibration data string from the current calibration parameters which will be sent to the client (see Chapter *Receive general data from the server* below on how to receive the data). The client should then save this data for future use.

Validation

You can check if the current device provides validation functionality for checking eye tracking quality. To request the call about validation availability, send a message with this JSON string:

```
{
      "action":"isValidationAvailable"
}
```

The response will be given in a form of the Notification with code 14 and with field *isValidationAvailable*.

To start a validation, send a message with this JSON string:

```
{
      "action":"setValidation",
      "validation":true
}
```

When the server receives this message, it will start a validation procedure. For now, the validation is only available with devices with *automatic without markers* calibration mode. If you select the *automatic with markers* or the *automatic without markers*, then the client will receive the points from the server to render that. The points will be sent with the same Notification as the calibration points. At the end of the validation the server will send the Notification (code 10) about the validation score: *verror* field.

Receive general data from the server

The server sends data back to the client in JSON string format. Server will respond to the receiver port 3430 (initial port address + 1) with the device list. The Notification system is

used to structure the server's messages. There are 6 types of notifications: Possible notifications are:

1. EyeServerStarted
2. Calibration,
3. ScenePicInfo,
4. Msg,
5. ErrorMsg,
6. Warning

Server will respond to the receiver port 3430 (initial port address + 1) with the notifications. Here we show three possible examples:

The server sends a notification that the eye-tracker device has started successfully and is running:

```
{
      "action":"sendNotification",
      "tag": "EyeServerStarted"
      "message":{
            "eyeServerStarted": "true"
      }
}
```

The server sends a notification about the size of the scene picture so the client can prepare for receiving and displaying them.

```
{
      "action":"sendNotification",
      "tag": "ScenePicInfo"
      "message":{
            "height": "480",
            "width": "640"
      }
}
```

The server sends notifications during the calibration process that contain the text message, number of accepted points, and the total number of calibration points.

```
{
      "action":"sendNotification",
      "tag": "Calibration"
      "message":{
            "message": "Click Calibration Target",
            "numberOfAcceptedPoints": "0",
            "numberOfCalibrationPoints": "9"
      }
```

```
}
```

The Msg notifications are used for a wealth of purposes. For the current API version we have 24 options and for each option there is a code. Here is an example:

```
{
      "action":"sendNotification",
      "tag": "Msg"
      "message":{
            "message": "Calibration point accepted",
            "code": "2"
      }
}
```

Possible codes:

Code	***Explanation***
1	There is an error with the watermark image, the system cannot run
2	Calibration point was accepted. The message contains the point ID
3	Calibration point was rejected. The message contains the point ID
4	Calibration is done
5	Eye-event Listener is registered
6	Error with eye-event listener
7	Eye-event Listener information
8	General information is supplied
9	Response to the client for rendering the calibration point
10	Validation result
11	General message about the calibration
12	General message about automatic calibration
13	Error message about automatic calibration
14	Response to the client about the availability of the validation process
15	Error message: Validation is not available but was requested

16	General message about the scene camera usage for the current device
17	General error about the scene camera use for the current device
18	List of the allowed/available calibration types
19	List of the allowed/available SeeTrue features
20	List of the EyeTracking devices
21	Calibration data from the current calibration
22	Error message about Gaze UI listeners
23	Error message about Marker tracking
24	Error message about Eye Images

For the error messages we use the ErrorMsg tag, e.g.:

```
{
      "action":"sendNotification",
      "tag": "ErrorMsg"
      "message":{
            "message": "There is an error with the watermark image"
            "code": "1"
      }
}
```

Receive eye data from the server

Eye data is sent to the Eye-Data Port 3428 by default (initial port address - 1). The server uses the ZeroMQ library *(https://zeromq.org/)* to handle this communication. It has a CSV encoded string format.

Eye data refers to the gaze data obtained during tracking. Each eye data sample contains the following metrics (‘CS’ refers to coordinate system):

1. Frame counter
2. Timestamp, counting milliseconds since the first data entry (whose timestamp is thus 0,0 ms)
3. Gaze point X in relative scene camera coordinates where image width = 1 and image height = 1; e.g., (0.7103 ; 0.4417). The absolute coordinates are computed by scaling / multiplying the relative coordinates with the image’s dimensions.
4. Gaze point Y.
5. Estimated pupil area for the right eye, given as square millimeters.
6. Estimated pupil area for the left eye, given as square millimeters.

7. Estimated pupil area for the right eye, given as pixels.
8. Estimated pupil area for left eye, given as pixels.
9. An acceptance measure (from 0 to 1).
10. An acceptance measure for the right eye (from 0 to 1).
11. An acceptance measure for the left eye (from 0 or 1).
12. Index of the scene image.
13. System Time, Year:Month:Day:Hour:Minute:Second:Milliseconds
14. X coordinate of the right cornea center,in the right eye camera CS [mm]
15. Y coordinate of the right cornea center,in the right eye camera CS [mm]
16. Z coordinate of the right cornea center,in the right eye camera CS [mm]
17. X coordinate of the left cornea center,in the left eye camera CS [mm]
18. Y coordinate of the left cornea center,in the left eye camera CS [mm]
19. Z coordinate of the left cornea center,in the left eye camera CS [mm]
20. Calibration Error (anything larger than 0).
21. Eye Event (String that will be explained later, in this section).

If the system includes the license for using markers, the *Eye data* is extended with information about detected markers. Up to four markers can be detected simultaneously. The information is as follows:

- ID of 1st...4th detected marker
- Gazepoint X in the coordinates of 1st...4th marker
- Gazepoint Y in the coordinates of 1st...4th marker
- Center point X of 1st...4th marker, in absolute scene coordinates
- Center point Y of 1st...4th marker, in absolute scene coordinates

NodeJS example:

```
const zmq = require("zeromq");
var isRun = true;
var sub = new zmq.Pull;
sub.receiveHighWaterMark = 1;
sub.connect("tcp://" + params.eyeServerId + ":" + params.eyeDataPort);
try {
      for await (const [msg] of sub) {
            if(!isRun){
                  break;
            }
            let stringMessage = new TextDecoder("utf-8").decode(msg);

            let subStrt = stringMessage.split(";");
            let gazeX = parseFloat(subStrt[2]) * sceneFrameWidth;
            let gazeY = parseFloat(subStrt[3]) * sceneFrameHeight;

            let scoreLeftEye = parseFloat(subStrt[9]);
            let scoreRightEye = parseFloat(subStrt[10]);

            let pupilAreaInMM2RightEye = parseFloat(subStrt[4]);
            let pupilAreaInMM2LeftEye = parseFloat(subStrt[5]);
      }
```

```
} catch (err) {
      console.log(err);
}
```

In this example we receive a message and decode it to UTF-8 format. Because the message has CSV type, we split the message with the ";" separator. The CSV data does not contain a header. The third and fourth values (indexing begins from 0) keep the relevant gazeX and gazeY coordinates. Gaze point X and Y values can be negative, or larger than 1 – meaning that the gaze location is outside the scene image (for instance on AR systems outside the screen).

The logic for The Eye Event string is as follows:

- "" - empty string when there is no identified event (the events trigger once and in between the string is empty
- "NA" - always when either eye is not available.
- "S" - always when either eye makes a saccade.
- "BB" - when either eye starts a blink.
- "BE" - when either eye finishes a blink.
- "FB" - when both eyes fixate for the first time during this fixation.
- "FEx0.123y0.234d5.678" when either eye finishes a fixation, where:
    - x0.123 is the relative x coordinate of the weighted x coordinates during the fixation, weights being the detection scores;
    - y0.234 is the relative y coordinate of the weighted y coordinates during the fixation, weights being the detection scores;
    - d5.678 is the duration of the fixation, in seconds.

In addition, if the eyes are blinking or are not present, the output gaze point is set to (0,0) and the pupil parameters (size etc.) are set to 0.

Receive eye-pixel frames from the server

This functionality is subject to the license admitting eye image streams.
The pixel data of each frame is sent to the picture ports accordingly. The eye pixel data is sent as a raw in one channel grayscale picture. The server uses ZeroMQ library (https://zeromq.org/) to handle this communication. The size of eye-pictures is currently 320 x 240 pixels. For the faster pixel processing in this example we used the Sharp library for NodeJS (https://github.com/lovell/sharp)

NodeJS example:

```
const zmq = require("zeromq");
var sub = new zmq.Pull;
var isRun = true;
sub.receiveHighWaterMark = 1;
sub.conflate = true;
```

```
sub.connect("tcp://" + params.eyeServerId + ":" +
params.leftEyePicServerPort);

try {
	for await (const [msg] of sub) {
		if (!isRun) {
			break;
		}
		let metaData = null;
		try {
			metaData = await sharp(msg, {
			raw: {
				width: Number(params.eyeFrameWidth),
				height: Number(.params.eyeFrameHeight),
				channels: 1
			}
			}).toColourspace('rgba').ensureAlpha().toBuffer({
resolveWithObject: true });
		} catch (err) {
			console.log(err);
		}
		if (metaData != null){
			// here you could pass metaData.data for your purposes
		}
	}
} catch (err) {
	console.log(err);
}
```

Scene pixel data is sent in JPEG format as a byte array. The first 4 bytes are used for the frame number encoding. This frame number is used for the synchronization with eye-data. The server used ZeroMQ library *(https://zeromq.org/)* to handle this communication. For the faster pixel processing in this example we used the Sharp library for NodeJS *(https://github.com/lovell/sharp)*

NodeJS example:

```
const zmq = require("zeromq");
var sub = new zmq.Pull;
var isRun = true;
sub.receiveHighWaterMark = 1;
sub.conflate = true;
sub.connect("tcp://" + params.eyeServerId + ":" +
params.scenePicServerPort);

try {
	for await (const [msg] of sub) {
		if (!isRun) {
			break;
		}
		let metaData = null;
		let jpegData = null;
		try {
```

```
                        var u8 = new Uint8Array([msg[0], msg[1], msg[2],
msg[3]]);
                        var view = new DataView(u8.buffer);
                        frame = view.getInt32();
                        jpegData = msg.slice(4);
                         metaData = await
sharp(Buffer.from(msg.slice(4))).raw().ensureAlpha().toBuffer({
resolveWithObject: true });
                } catch (err) {
                        console.log(err);
                }
                if (metaData != null) {
                        // here you could pass metaData.data and frame for your
                purposes
                }
        }
} catch (err) {
        console.log(err);
}
```

Eye-event listeners

The SeeTrueEyeServer provides registration of the eye-event listeners. The eye-event listener is an interactive area that has various states depending on the gaze location and gaze behavior. With eye-event listeners, you can make a gaze-contingent graphical user interface quickly and cleanly. The current API allows you to add a new eye-event listener, remove it, update the listener, get the current status of the listener, and remove all listeners from the server.

To register a new eye-event listener, you should send a message with this JSON string:

```
{
        "action":"addGazeListener"
        "shape":{
                "type":"rect",
                "topLeftX":0.08191149635036499,
                "topLeftY":0.35581614170168757,
                "width":0.20875912408759123,
                "height":0.3304007030087099
        },
        "dwellTime":500,
        "type":"basic",
        "id":-1,
}
```

The eye-event listener has a shape, where you describe the area. The current API has the rectangular shape only. You set topLeftX and topLeftY coordinates of the rectangle and its

width and height. All values should be formed in relative values. That is, the top left corner of the scene image is (0,0) and bottom right corner is (1,1). Also, it is possible to register listeners outside of the given range. For example, if you want to place the button outside of the scene view (an off-screen glance target) you can register eye-event listener as follows:

```
{
      "action":"addGazeListener"
      "shape":{
            "type":"rect",
            "topLeftX":-0.5,
            "topLeftY":-0.5,
            "width":0.1,
            "height":0.05
      },
      "dwellTime":500,
      "type":"basic",
      "id":0,
}
```

For now, the type is always basic for the current API version. In the future, this will affect the calculation method which is used on the server to trigger the event on the listener. According to the basic type, the event happens when the user gaze is located inside the area more than the dwellTime (milliseconds).
When you register an eye-event listener, you should provide it with a listener ID. The ID is a positive integer number. If you set the ID to 0, the server will give the ID by itself. If you provide the ID that is already occupied by another listener, the listener will not be registered.

When the eye-event listener is registered, the server sends the notification with code = 5 about that:

```
{
      "action":"sendNotification",
      "tag": "Msg"
      "message":{
            "code": "5",
            "message": "{
                "dwellTime": "500",
                "id": "1",
                "state": "none_active",
                "type": "basic",
                "shape": {
                    "type": "rect",
                    "topLeftX": "0.175342157",
                    "topLeftY": "0.281524867",
                    "width": "0.573722601",
```

```
                    "height": "0.48484838"
                }
            }"
        }
}
```

You can find the ID that was used to register the listener and its parameters.

The server makes a new empty list for eye-event listeners in each run, which means that you should register listeners each time as well. The client software should take care of this at initialization.

When an eye-event listener is triggered, the server sends a message to the Eye-event Listeners Port (initial port address - 6, see the Opening Ports section of the current doc for more details). Here is an example message:

```
{
        "id": "1",
        "state": "active"
}
```

…communicating that the eye-event listener with ID = 1 is triggered.

Eye-event listeners can be removed with the following message (for example, removing listener with ID 5):

```
{
        "action" : "removeGazeListener",
        "id" : 5
}
```

To update an eye-event listener you should specify the listener properties and send it to the server with a valid ID. An already used ID will overwrite the previous listener:

```
{
        "shape":{
                "type":"rect",
                "topLeftX":0.06293339416058397,
                "topLeftY":0.2072335770350488,
                "width":0.8437956204379562,
                "height":0.5161289088420082
        },
        "dwellTime":"100",
        "type":"basic",
        "id":1,
```

```
      "action":"updateGazeListener"
}
```

To get the current status of the listener, you should send this message to the server:

```
{
      "action" : "getStatusGazeListener",
      "id" : 5
}
```

When the server receives this message, it will try to find the eye-event listener with the given ID and send the notification back to the client with code = 7:

```
{
      "action": "sendNotification",
      "tag": "Msg",
      "message": {
            "message": "{
                  "dwellTime": "500",
                  "id":"1",
                  "state":"none_active",
                  "type":"basic",
                  "shape":"{
                        "type": "rect",
                        "topLeftX": "0.187020987",
                        "topLeftY": "0.224828884",
                        "width":"0.540145993",
                        "height":"0.588465154"
                  }"
            }",
            "code": "7"
      }
}
```

To remove all eye-event listeners, you should send this message to the server:

```
{
      "action" : "removeAllGazeListeners"
}
```

The eye-server alive message

To check if the server is alive, you could use the ping message:

```
{
```

```
        "action" : "ping",
        "msg" : "Are you here?"
}
```

The server will receive the ping message and send it back to the client via notification with code=8:

```
{
        "action":"sendNotification",
        "tag": "Msg"
        "message":{
                "code": "8",
                "message": "Are you here?"
        }
}
```

Add-on functionalities

A set of additional and helper functions can be implemented and included for various customer applications. These features are turned on and off and configured before deployment.
These functionalities work under a specific license for each function.

Marker tracking

The marker tracking enables one to get the gaze point in each detected marker's own coordinate system. These are outputted at the end of the Eye data.
To start the marker tracker algorithm, send the following message:

```
{
        "action" : "startMarkerTracking"
}
```

To stop the marker tracker algorithm, send the following message:

```
{
        "action" : "stopMarkerTracking"
}
```

Custom scene camera/no scene camera

By default, the system is configured to use a predefined scene camera. External camera, for example from a client's hardware, can be configured to be used instead. Please consult our contact person.

Appendix 1.

The following describes the marker JPEGs:

```
let markerList = [

"
BAQECAgICAgQDAgICAgUEBAMEBgUGBgYFBgYGBwkIBgcJBwYGCAsICQoKCgoKBggLD
AsKDAkKCgr/wAALCABkAGQBAREA/8QAHwAAAQUBAQEBAQEAAAAAAAAAAAECAwQFBgc
ICQoL/8QAtRAAAgEDAwIEAwUFBAQAAAF9AQIDAAQRBRIhMUEGE1FhByJxFDKBkaEII
0KxwRVS0fAkM2JyggkKFhcYGRolJicoKSo0NTY3ODk6Q0RFRkdISUpTVFVWV1hZWmN
kZWZnaGlqc3R1dnd4eXqDhIWGh4iJipKTlJWWl5iZmqKjpKWmp6ipqrKztLW2t7i5u
sLDxMXGx8jJytLT1NXW19jZ2uHi4+Tl5ufo6erx8vP09fb3+Pn6/9oACAEBAAA/AP5
/6KKKKKKKKKKKKKKKKKKK9//wCHTv8AwVN/6Rp/tAf+Gb1z/wCRawPil/wT2/b6+
B3gS++KXxr/AGHvjB4P8MaX5X9p+I/FPw01XT7C08yVIY/NuJ7dI498skca7mG5nVR
kkCvH6KKKKKKKKK/v8r4A/4Ojv8AlBR8c/8AuWf/AFJ9Jr+QKiiuw/Z6+Cnir9pT4
++B/wBnPwLqGn2mt+P/ABhpnhvRrrVpXjtYbq+uo7WJ5mjR3WIPKpYqjsFBIVjwf1O
/4gqf+Cpv/RfP2f8A/wAKnXP/AJT0f8QVP/BU3/ovn7P/AP4VOuf/ACno/wCCIKn/gq
b/0Xz9n/wD8KnXP/lPXwh/wVA/4Jf8Ax9f8AJ/Wtb1rwfb+JLW68
FahdXNqlrNdXdqqO11bW7iUPZykgIV2sh3EkgfOFFFf0/f8Rq3/BLL/ogf7QH/hLaH
/8ALivl/wD4LM/8HQP7Av8AwUS/4JsfEj9jr4KfL4LM/8HQP7Av8AwUS/4JsfEj9jr
JrFjfSea8GpTSLmK2kC7Y2yxUHAJYfhDRRXv/APwSd/5Sm/s0/wDZwHg3/wBPlnX9v
tFFfzA/8Hq3/KU3wD/2b/pX/p81yvyBoooooooorsP2evjX4q/Zr+Pvgf9ozwLp+n3e
t+APGGmeJNGtdWieS1murG6juokmWN0doi8ShgroxUkBlPI/U7/iNW/4Km/8ARA/2f
/8Awltc/wDlxR/xGrf8FTf+iB/s/wD/AIS2uf8Ay4o/4jJvif/Cv/RA/2f8A/wAJbXP
/AJcV8If8FQP+CoHx9/4KzfH3SP2jP2jPCHg/Rdb0Xwfb+G7W18FafdW1q9rDdXd0r
ut1c3DmUveSgkOF2qg2ggk/OFFFf0/f8QVP/BLL/ovn7QH/AIVOh/8AhD/E+FqQv
dyMrLnKkEA1/U7/AMQuP/BCj/oxn/zJvif/AOWdfnB/wdA/8EZdT9EvtT/AOEw1jUPNsJNK1Wd4fLvryaNcy20DbgoYbMAgFgfw
hr93v8Ag1+/4Izf8E2P+CiX7Avi/4Izf8E2P+CiX7Avi/4Izf8AB0d/ygo+Of8A3LP
/AKk+k1/IFRRXv/8AwSd/5Sm/s0/9nAeDf/T5Z1/b7X5A/wDB6t/yiy8A/wDZw
Glf+mPXK/mBr+n7/gyp/wCUWXj7/s4DVf8A0x6HX6/UUUV8Af8AB0d/ygo+Of8A3LP
/AKk+k1/IFRRRRRRX9P3/AAZU/wDKLLx9/BnAar/AOmPQ6/X6iiv4A6KKKKKKK/d
7/g1+/4LM/8E2P+Cdv7Avi/4Kfti/tIf8If4n1T4wahrdjpn/CH6xqHm2EmlaVAk3m
WNnNGuZbaddpYMNmSACpP6P8A/EUd/wAEKP8Ao+b/AMxl4n/+VlH/ABFHf8EKP+j5
/MZeJ//AJWUf8RR3/BCj/o+b/zGXif/AOVlfyBUUUUUUUUUUUUUUUUUUUUUUUUUU
UV//Z",
```

"
BAQECAgICAgQDAgICAgUEBAMEBgUGBgYFBgYGBwkIBgcJBwYGCAsICQoKCgoKBggLD
AsKDAkKCgr/wAALCABkAGQBAREA/8QAHwAAAQUBAQEBAQEAAAAAAAAAAAECAwQFBgc
ICQoL/8QAtRAAAgEDAwIEAwUFBAQAAAF9AQIDAAQRBRIhMUEGE1FhByJxFDKBkaEII
0KxwRVS0fAkM2JyggkKFhcYGRolJicoKSo0NTY3ODk6Q0RFRkdISUpTVFVWV1hZWmN
kZWZnaGlqc3R1dnd4eXqDhIWGh4iJipKTlJWWl5iZmqKjpKWmp6ipqrKztLW2t7i5u
sLDxMXGx8jJytLT1NXW19jZ2uHi4+Tl5ufo6erx8vP09fb3+Pn6/9oACAEBAAA/AP5
/6KKKKKKKKKKKK0PCfhPxV498VaZ4F8C+GdQ1rW9a1CGw0bRtJsnubq/upnEcVvDF
GC8sruyoqKCzMwABJr2/wD4dO/8FTf+kaf7QH/hm9c/vPBU3/pGn+0B/4Zv7i5u
XP/kWj/h07/wVN/6Rp/tAf+Gb1z/vPBU3/pGn+0B/4Zv7i5u
a6N418M3WlXU1qzyRrcJFdRo7RF4pUDgbS0bjOVOOPr2D4W/wDBPb9vr44+BLH4pfB
T9h74weMPDGqeb/ZniPwt8NNV1Cwu/LleGTyriC3eOTZLHJG21jtZGU4IIrf/AOHTv
/BU3/pGn+0B/wCGb1z/AORa8Aoooor3/wD4JO/8pTf2af8As4Dwb/6fLOv7faKK/mB
/4PVv+UpvgH/s3/Sv/T5rlfkDX9fv/Brj/wAoKPgZ/wBzN/6k+rV9/wBfwB0UUUV7/
wD8Enf+Upv7NP8A2cB4N/8ATt/wb0/tof8ABWb9tDwx+0Z+zn8
TvhfouiaL8L7Lw3dWvjXWtRtrp7qHUdSumdFtbC4QxFLyIAlw25XG0AAn4P8A+IKn/
gqb/wBDF8/Z//lZXyB/wXq/4L1f8Env20f8Agk98Vv2aP2aP2rP+El8beJf7C/sTRP8
AhBd
wDMZeJ//lZXyB/wXq/4L1f8Env20f8Agk98Vv2aP2aP2rP+El8beJf7C/sTRP8AhBd

ds/tP2fXdPupv311YxQpthglf5nGduBliAf5waKKKKKKKKKKKKKKKKKKKKKKKKK/
9k=",

"
BAQECAgICAgQDAgICAgUEBAMEBgUGBgYFBgYGBwkIBgcJBwYGCAsICQoKCgoKBggLD
AsKDAkKCgr/wAALCABkAGQBAREA/8QAHwAAAQUBAQEBAQEAAAAAAAAAAAECAwQFBgc
ICQoL/8QAtRAAAgEDAwIEAwUFBAQAAAF9AQIDAAQRBRIhMUEGE1FhByJxFDKBkaEII
0KxwRVS0fAkM2JyggkKFhcYGRolJicoKSo0NTY3ODk6Q0RFRkdISUpTVFVWV1hZWmN
kZWZnaGlqc3R1dnd4eXqDhIWGh4iJipKTlJWWl5iZmqKjpKWmp6ipqrKztLW2t7i5u
sLDxMXGx8jJytLT1NXW19jZ2uHi4+Tl5ufo6erx8vP09fb3+Pn6/9oACAEBAAA/AP5
/6KKKKKKKKKKK0PCfhPxV498VaZ4F8C+GdQ1rW9a1CGw0bRtJsnubq/upnEcVvDF
GC8sruyoqKCzMwABJr2/wD4dO/8FTf+kaf7QH/hm9c/+Ra5D41/sRftofs1+Fbfx1+
0Z+yJ8UPAGiXeoJYWus+NfAGo6VazXTJJItukt1CiNKUilcIDuKxucYU48voooooo
oor3/8A4JO/8pTf2af+zgPBv/p8s6/t9r8gf+D1b/lFl4B/7OA0r/0x65X8wNFFFFF
FFFFeofsRfGvwr+zX+2h8If2jPHWn6dd6J4A+KGgeJNZdJiSS6mtbHUYLqVIVkdEa
UpEwUM6KWIBZRyP6Lv+I1b/AIJZf9ED/aA/8JbQ/wD5cV8H/wDBwt/wcL/ABWb
9i/wx+zn+zn8Mfihout6L8ULLxJdXXjXRdOtrV7WHTtStWRGtb+4cyl7yIgFAu1XO4
EAH8ca+7/8Agl//AMG9P7aH/BWb4Bav+0Z+zn8TvhihIf2jPHWn6hd6J4A+KGgeJN
bW0umdFtbC4QxFLyIAlw25XG0AAn6P/4gqf8Agqb/ANF8/Z//APCp1z/5T0f8QVP/A
AVN/wCi+fs//wDhU65/8p6P+KGqb/wBF8/Z//APCp1z/AOU9H/EFT/wVN/6L5+z
/AP8AhU65/wDKevIP29f+DX79vr/gnb+yd4r/AGxfjX8Xfg/qnhjwf9h/tOx8La/qs
9/L9rv7exj8pJ9NhjbEtzGW3SLhQxGSAp/OCiiiiiv6fv+DKn/AJRNsMEr/M4ztwMsQ
D/ODRRRRRRRRRRRRRRRRRRRRRRRRRRX//Z",

"
BAQECAgICAgQDAgICAgUEBAMEBgUGBgYFBgYGBwkIBgcJBwYGCAsICQoKCgoKBggLD
AsKDAkKCgr/wAALCABkAGQBAREA/8QAHwAAAQUBAQEBAQEAAAAAAAAAAAECAwQFBgc
ICQoL/8QAtRAAAgEDAwIEAwUFBAQAAAF9AQIDAAQRBRIhMUEGE1FhByJxFDKBkaEII
0KxwRVS0fAkM2JyggkKFhcYGRolJicoKSo0NTY3ODk6Q0RFRkdISUpTVFVWV1hZWmN
kZWZnaGlqc3R1dnd4eXqDhIWGh4iJipKTlJWWl5iZmqKjpKWmp6ipqrKztLW2t7i5u
sLDxMXGx8jJytLT1NXW19jZ2uHi4+Tl5ufo6erx8vP09fb3+Pn6/9oACAEBAAA/AP5
/6KKKKKKKKKKKK0PCfhPxV498VaZ4F8C+GdQ1rW9a1CGw0bRtJsnubq/upnEcVvDF
GC8sruyoqKCzMwABJr2/wD4dO/8FTf+kaf/HhW498Vaz4F8C+GdQ1rW9a1CGw0bRtJsnubq/upnEcVvDF
XP/kWj/h07/wVN/6Rp/tAf+Gb1z/5FrzD41/s9fH39mvxVb+Bf2jPgd4w8Aa3d
a6N418M3WlXU1qzyRrcJFdRo7RF4pUDgbS0bjOVOOPr2D4W/wDBPb9vr44+BLH4pfB
T9h74weMPDGqeb/ZniPwt8NNV1Cwu/LleGTyriC3eOTZLHJG21jtZGU4IIrf/AOHTv
/BU3/pGn+0B/wCGb1z/AORa8Aoooor3/wD4JO/8pTf2af8As4Dwb/6fLOv7faKK/mB
/4PVv+UpvgH/s3/Sv/Brj/wB0KPgZ/wBzN/6k+rV9/wBfwB0UUUV7/
wD8Enf+Upv7NP8A2cB4N/8AzZ1/b7RRX44/wDBwt/wb0/tof8ABWb0D/iNW/wCCOD
TvhfouiaL8L7Lw3dWvjXWtRtrp7qHUdSumdFtbC4QxFLyIAlw25XG0AAn4P8A+IKn/
gqb/wBF8Z/Z//wDCp1z/AOU9fYH7Ln/Bav2+Ad/Ag/Ab/Ag/Ab/Ag/Ab/Ag/Ab/
9q/4SbXfhbpVje6Fc/wBq3Uut2/2Wa/vLO4fbbalAj77ePEqSBd6hXb0D/iNW/wCCW
X/RA/2gP/CW0P8A+XFfn/8A8QVP/BU3/ovn7P8A+XFfn/8A8QVP/BU3/ovn7J3iv8AbC
dv7J3iv8AbF+Nfxd+D+qeGPB/2H+07Hwtr/t7GPykn02GNsS3MZbdIuFDE
ZICn84KKK9//wCCTv8AylN/Zp/7OA8G/wDp8s6/t9oooor+QL/g6O/gvH/uWf/
Y0mvgCv7/ACvgD/g6O/gvH/uWf/Un0mv5AqKK7D9nr41+Kv/j74H/
AZ/8A/CW1b/wDQ8T/s5/tC/HH4X/wDQ8T/s5/tC/HH4X/wDQ8T/s5/tC/HH4X/
Uba6e6h1HTbVUdrq/uEMRS8lJAQNuVDuABB/Y6iiv5Av+Do7/lOv8c/+5Z/9RjSa+A
K/v8AK+AP+Do7/lBR8c/+5Z/9SfSa/kCoooor9fv+DKn/AJSc/
IFRRRRX6/f8GVP/KU3x9/2b/qv/p80Ov6fq/CH/g6B/wCCzP8AwUn/
sdftIf8If4Y1T4P6frd9pn/AAh+j6h5t/JquqwPN5l9ZzSLmK2gXaGCjZkAEsT+cH/
EUd/wXX/6Pm/8xl4Y/wDlZXyB+1H+1H8dv20fjtrv7S/7S/jn/hHJfG3iX7L/bet/2Z
a2f2n7PaxWsP7m1iihTbDBEnyoM7cnLEk+f1/f5XwB/wdHf8dHf5KPjn/mB/wCD1b
Ff3+V8AR3/KCj45/mB/wCD1b/lKb4B/wCD1b/mB/wCD1b/lKb4B/wCD1b/mB/wCD1
Ff3+V8Af8HR3/KCj45/9yz/AOpPsvH3ZwGq/+PQ6/X8Ag
p+2L+0h/wAIf4n1T4wahrdjpn/CH6xqHm2EmlaVAk3mWNnNGuZbaddpYMNmSACpP6P
/APEUd/wQo/6Pm/8AMZeJ/wD5WV/OD/wXq/sL+bet/4Bp/sL+xq/aj+BP7Ag
4l/sL+xNb/sy6s/tP2fQtPtPtZv3N1FFMm2aCVPmQZ25GVIJ+QKKKKKKKKKKKKKKKK
KKKKKKKKKKKK/9k=",
      ];

//The marker positioning with ThreeJS library

      for (let i = 0; i < 4; i++) {

```
            let image = new Image();
            image.src = markerList[i];
            let tmpTextureMarker = new THREE.Texture();
            tmpTextureMarker.image = image;
            image.onload = function () {
          tmpTextureMarker.needsUpdate = true;
            };

            let spriteMaterialMarker = new THREE.SpriteMaterial({
          map: tmpTextureMarker,
            });

            let markerSprite = new
THREE.Sprite(spriteMaterialMarker);
            let tmpScale = 1*(window.innerHeight /
window.innerWidth)*0.5; // That is 0.25 from the
window.innerHeight

            markerSprite.scale.set(tmpScale, tmpScale *
(window.innerWidth / window.innerHeight), 1);
            markerSprite.center = new THREE.Vector2(0,1);

            if(i == 0){
          markerSprite.position.set(1-tmpScale*0.5,
1-tmpScale*0.5*(window.innerWidth / window.innerHeight), 1);
            } else if(i == 1){
          markerSprite.position.set(-1+3*tmpScale*0.5,
1-tmpScale*0.5*(window.innerWidth / window.innerHeight), 1);
            } else if(i == 2){
          markerSprite.position.set(1-tmpScale*0.5,
-1+3*tmpScale*0.5*(window.innerWidth / window.innerHeight), 1);
            } else {
          markerSprite.position.set(-1+3*tmpScale*0.5,
-1+3*tmpScale*0.5*(window.innerWidth / window.innerHeight), 1);
            }

            calVisPointsSpriteGroup.add(markerSprite);
      }
```